UNE QUESTION AVANT DE COMMENCER

Où se cache ce que sait une IA ?

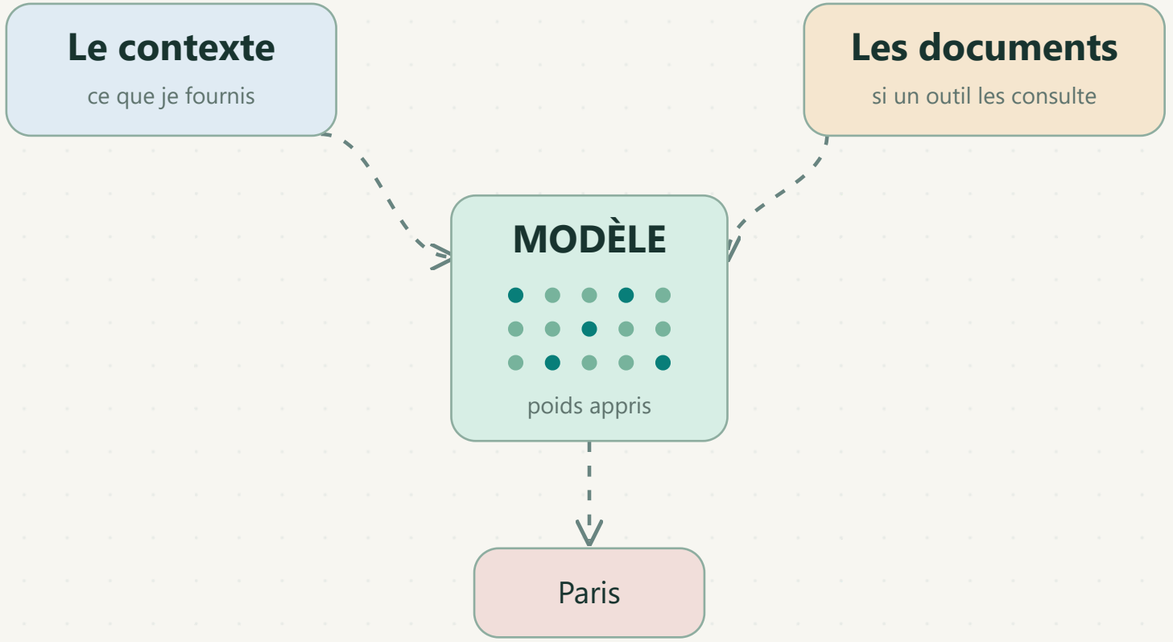
Un modèle complète : « La capitale de la France est… »

> A-t-il ouvert un dictionnaire ?

> A-t-il gardé une phrase ? Ou appris des réglages ?

À RETENIR

Pour répondre, distinguons les poids, le contexte et les documents consultés.



Une réponse peut mobiliser plusieurs sources.

Trois endroits différents. Ils peuvent coopérer dans une application.

DEUX MOMENTS À NE PAS CONFONDRE

Apprendre. Puis utiliser.

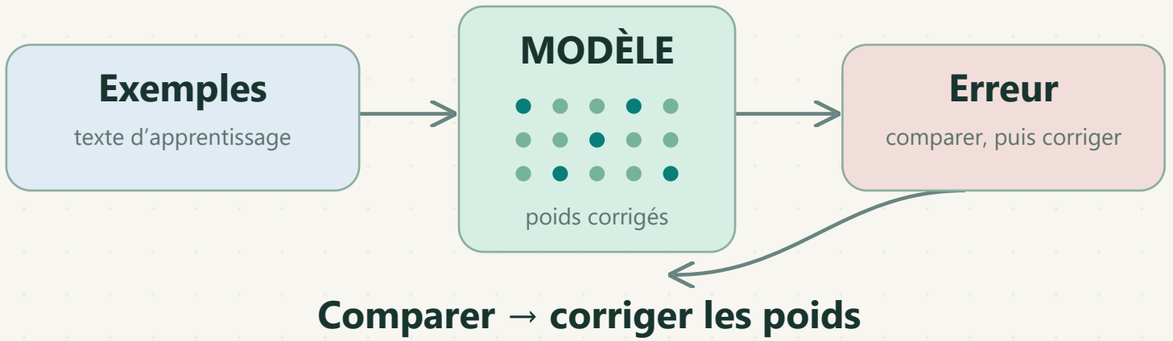
Pendant l'apprentissage, les poids changent.
Pendant une inférence ordinaire, ils restent fixes.

> **Poids :** les nombres qui règlent les calculs.

> **Contexte :** le texte fourni pour cette réponse.

À RETENIR

Lire un message n'est pas, en général, réentraîner le modèle.



ENSUITE : UNE INFÉRENCE

nouveau contexte → poids fixes → sortie

Schéma simplifié. Une application peut garder un historique sans modifier les poids.

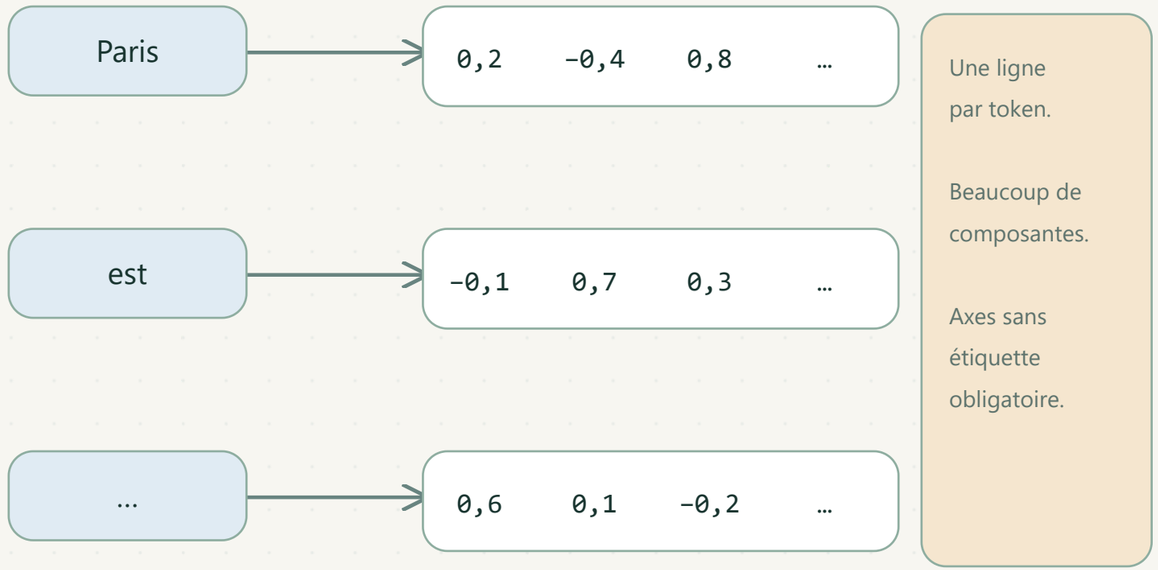
DU TEXTE AUX NOMBRES

Un morceau de texte devient un vecteur.

Un token est une unité de texte. Son embedding est une liste de nombres apprise.

> Le découpage dépend du tokenizer. Un token n'est pas toujours un mot.

> Un axe ne signifie pas forcément « joie », « animal » ou « passé ».



Tokens et coordonnées dessinés pour expliquer ; ce ne sont pas les sorties d'un tokenizer ou d'un LLM.

À RETENIR

Ces nombres deviennent utiles par les transformations du réseau et le contexte.

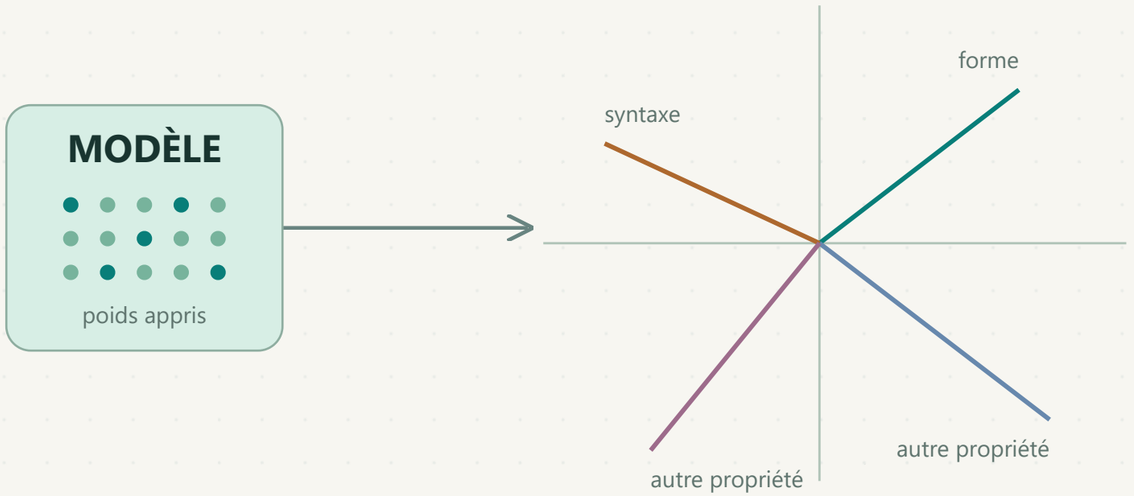
UNE REPRÉSENTATION PARTAGÉE

Un neurone n'est pas un tiroir à faits.

L'attention relie les tokens. Le MLP, un petit réseau, transforme chaque vecteur avec des matrices et une activation.

> Certaines associations sont localisables par des expériences causales.

> Des caractéristiques peuvent aussi partager des dimensions : c'est la superposition.



Plusieurs directions dans le même espace.

À RETENIR

Distribué ne veut pas dire incompréhensible, ni incapable de mémoriser un passage.

Directions schématiques. La superposition peut créer des interférences ; ce n'est pas un rangement parfait.

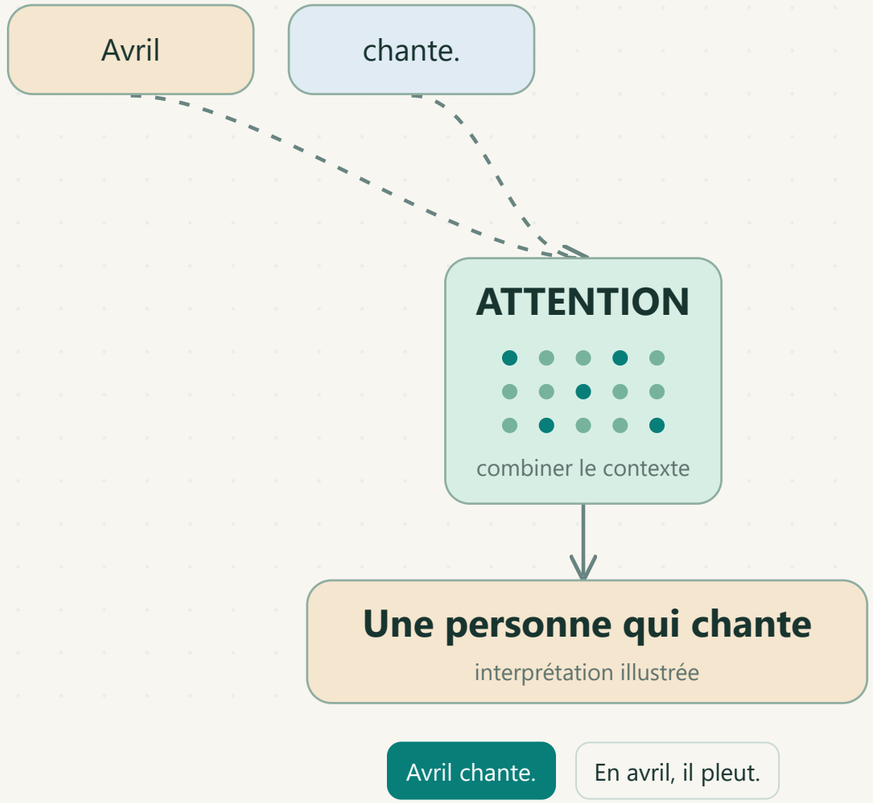
LE CONTEXTE CHANGE L'INTERPRÉTATION

« Avril » : qui ou quand ?

L'attention combine des informations du contexte.
Les couches suivantes les transforment encore.

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

Q : requêtes ; K : clés ; V : valeurs.
d_k : nombre de composantes d'une clé ; ^{T} : transposée.
softmax transforme les scores en pondérations positives de somme 1.



Les liens sont dessinés, pas mesurés dans un réseau. Dans un décodeur causal, le futur est masqué.

À RETENIR

L'attention route des informations. Elle n'est pas, à elle seule, un moteur de vérité.

À VOUS DE PRÉDIRE

J’ajoute un nom dans mon message.

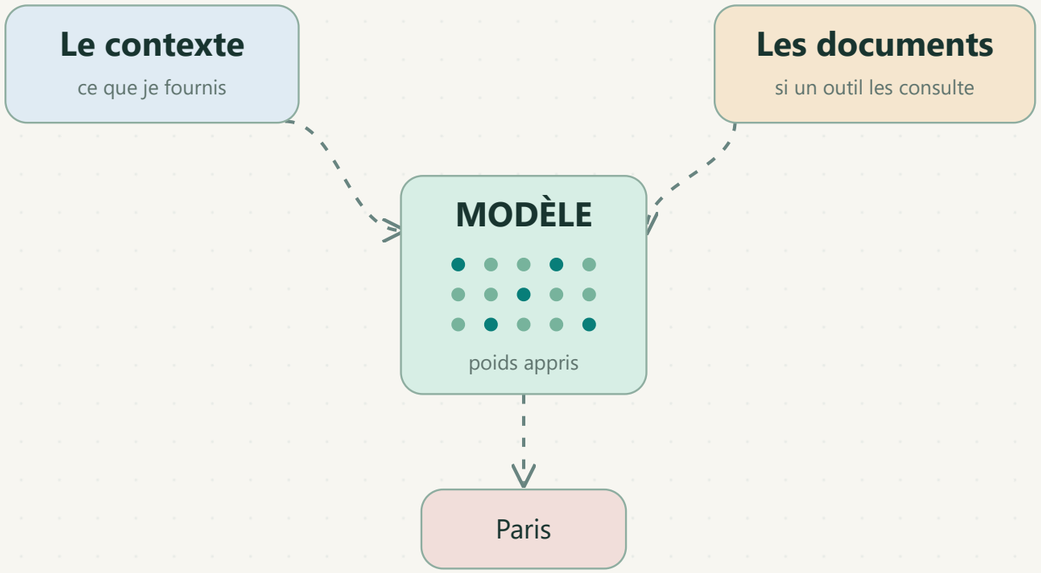
Le modèle le réutilise dans sa réponse. Qu’est-ce que cela démontre ?

A. Ses poids ont nécessairement changé.

B. Il peut exploiter le contexte fourni.

C. Ce nom restera appris pour tous les utilisateurs.

La réponse peut utiliser le contexte. À elle seule, elle ne prouve ni mise à jour des poids ni mémoire durable.



Une réponse peut mobiliser plusieurs sources.

À RETENIR

Poids appris, contexte temporaire et stockage de l’application sont trois choses différentes.

Une réponse correcte ne suffit pas à identifier le mécanisme de mémorisation.

DEUX PHASES DE L'INFÉRENCE

Lire le début. Puis continuer.

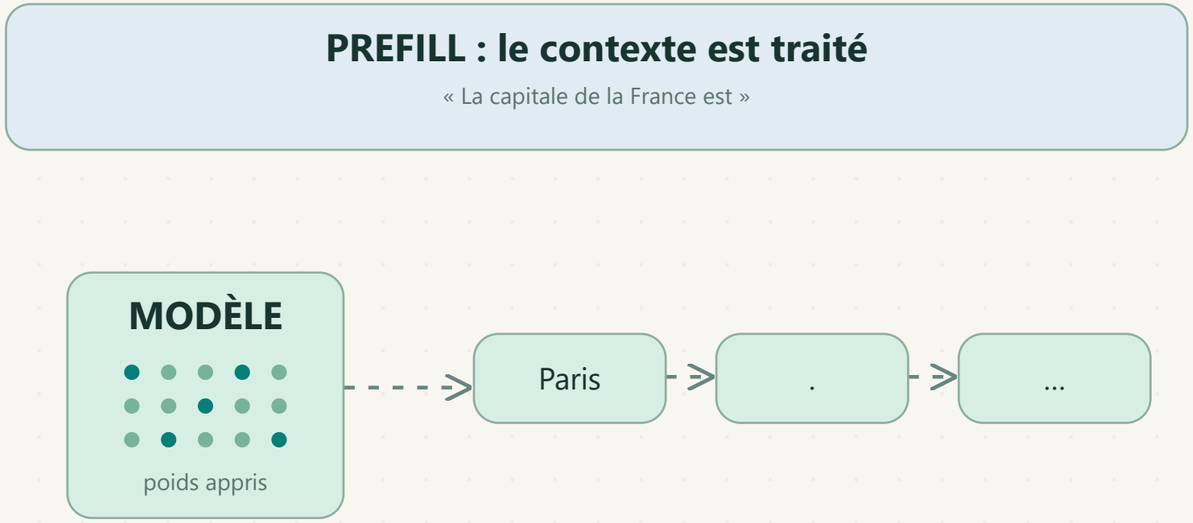
Prefill : traiter le contexte. Decode : produire la suite, token après token.

> Le calcul à l'intérieur d'une étape peut être parallélisé.

> En génération ordinaire, le token suivant dépend des précédents.

À RETENIR

Le coût du premier token et le débit de la suite sont deux mesures différentes.



DECODE : une suite qui se construit.

Génération autorégressive ordinaire. La spéculation, vue plus loin, peut vérifier plusieurs propositions ensemble.

PREMIER BUDGET : LES POIDS

Sept milliards de nombres.

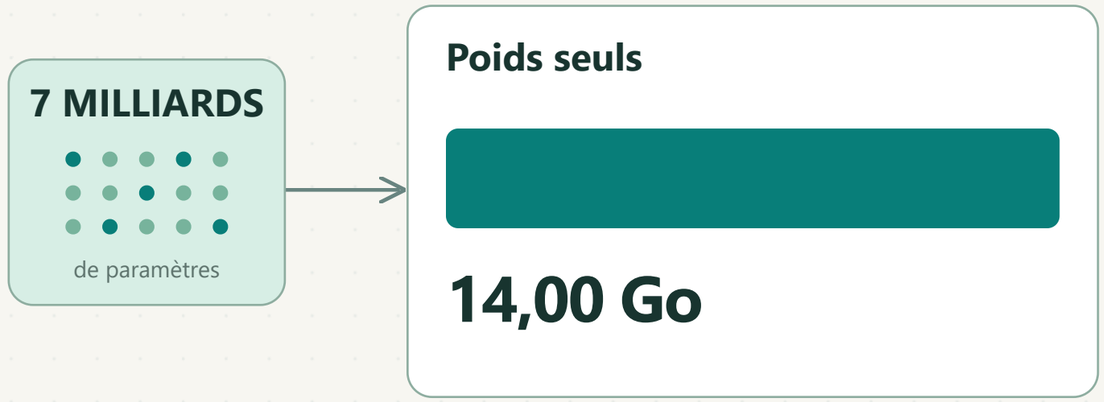
Moins de bits par poids, c’est moins de données à stocker et, potentiellement, à déplacer.

$$M_{poids} = N \times b / 8$$

N : nombre de paramètres ; b : bits par paramètre.
M : taille en octets. Ici, $N = 7 \times 10^9$.
1 Go = 10^9 octets.

À RETENIR

Le fichier réel ajoute des échelles, des métadonnées et parfois des tenseurs de précision différente.



16 bits par paramètre · compte idéal

16 bits | 8 bits | 4 bits | 2 bits

Compte théorique des poids seuls. Ce n’est ni la RAM totale ni un fichier GGUF réel.

DEUXIÈME BUDGET : LE CONTEXTE

Le cache grandit avec l'histoire.

Le cache K/V garde des clés et des valeurs pour éviter de recalculer le passé.

$$M_{KV} = 2 \times L \times S \times H_{KV} \times d \times p$$

Pour une séquence : L couches, S tokens,
H_{KV} têtes K/V, d composantes par tête,
p octets par composante. Le 2 compte K et V.



8 192 tokens · 8 têtes K/V

À RETENIR

Partager les têtes K/V, comme en GQA, réduit ce cache. Les valeurs doivent toujours être représentées.

Tokens 8 192 · 8 têtes K/V · changer

Exemple calculé : L=32, d=128, p=2. Cache non quantifié, sans surcoûts d'allocation ; 1 Gio = 2^{30} octets.

LE CALCUL ATTEND PARFOIS LES DONNÉES

Une usine rapide. Une route étroite.

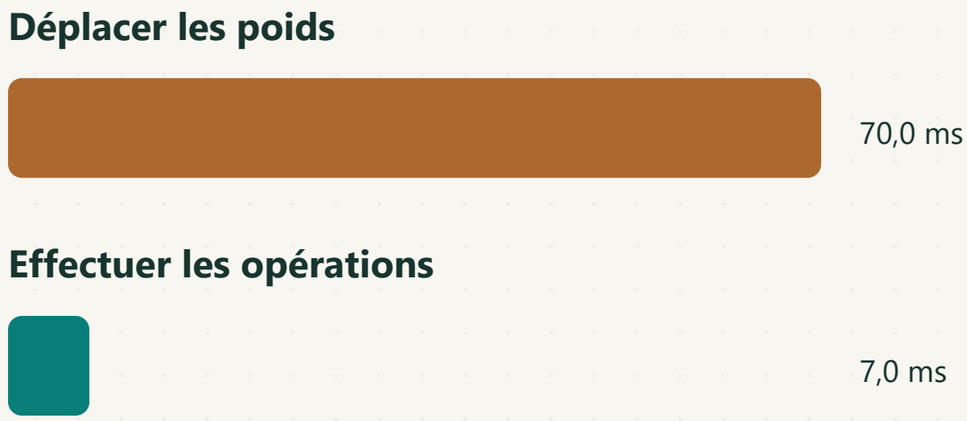
Le temps dépend aussi des octets qui arrivent au processeur, pas seulement du nombre de calculs.

$$T \geq \max(D / B, F / C)$$

D : octets transférés ; B : octets/s disponibles.
F : opérations ; C : opérations/s disponibles.
Ce modèle donne une borne optimiste.

À RETENIR

Le decode d'un modèle dense, avec une seule séquence, est souvent limité par la mémoire. Pas toujours.



BORNE OPTIMISTE ≥ 70,0 ms / étape

Au plus 14,3 tokens/s dans ces seules hypothèses

Débit mémoire — 50 Go/s — Poids 4 bits · changer

Modèle idéal : $N=7\times10^{9}$, D=poids seuls par étape, $F\approx2N$, $C=2\times10^{12}$ opérations/s. KV, activations et surcoûts exclus.

UN DIAGNOSTIC AVANT UNE OPTIMISATION

Le calcul prend 7 ms. La lecture, 70 ms.

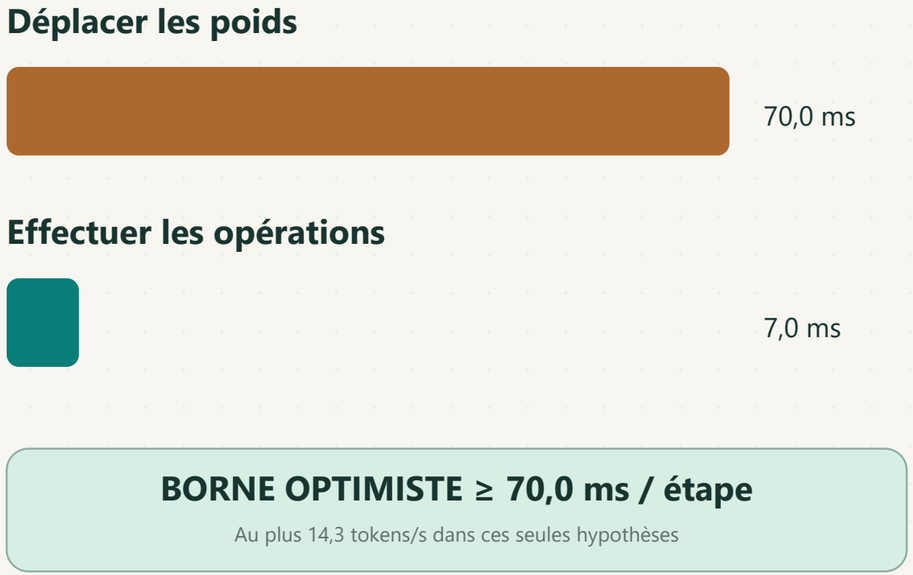
Dans cette situation idéale, doubler seulement la puissance de calcul donne-t-il un grand gain ?

A. Oui : la latence sera divisée par deux.

B. Non : les transferts restent la limite.

C. On peut annoncer exactement 140 tokens/s.

La borne max(70,7) reste 70 ms quand 7 devient 3,5 ms. Une machine réelle ajoute d'autres coûts.



Nombres d'un exemple analytique, pas d'une mesure de PC ou de Raspberry Pi.

À RETENIR

Optimiser une brique peu coûteuse peut presque ne rien changer au temps total.

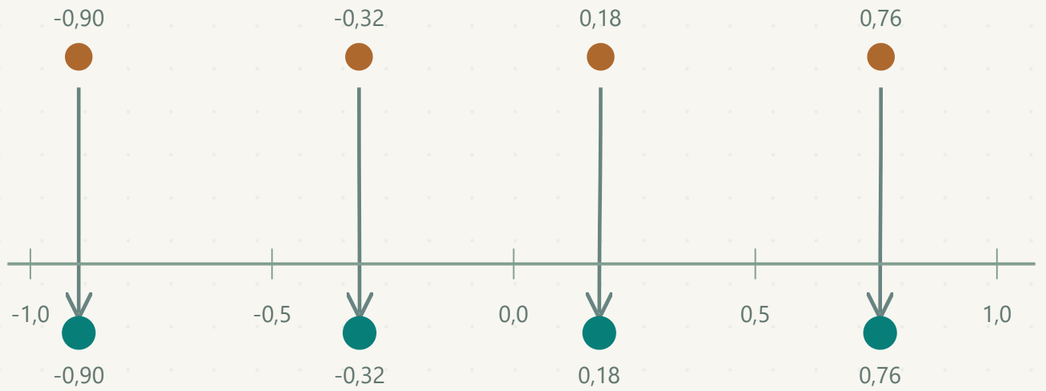
PISTE 1 · REPRÉSENTER AUTREMENT

Arrondir les poids. Observer la sortie.

Quantifier, c'est remplacer des valeurs par un petit ensemble de niveaux.

$$s = \max|w| \,/\, Q$$
$$q = \text{borne}(\text{arrondi}(w/s), -Q, Q)$$
$$\hat{w} = s \times q$$

$Q=2^{b-1}-1$; b : nombre de bits ; s : échelle.
w : poids ; q : code ; ŵ : poids reconstruit.
x : entrée ; y : sortie ; i : indice.
Si tous les poids sont nuls : s=1.



Sortie initiale : -1,0500

$y = \Sigma\, w_i x_i$

Après : -1,0470

écart absolu : 0,0030

8 bits | 4 bits | 2 bits

Produit scalaire jouet : w=(−0,9 ; −0,32 ; 0,18 ; 0,76), x=(1 ; 0,5 ; −1 ; 0,25). Arrondi à mi-distance loin de zéro.

À RETENIR

Petit fichier ne garantit ni petite erreur ni accélération. Ici, le calcul reste en flottant.

PISTE 2 · CHANGER LE TRAJET DES DONNÉES

Même attention. Moins d’allers-retours.

En 2022, Tri Dao et ses coauteurs proposent FlashAttention : organiser le calcul par blocs.

> Réutiliser des données dans la mémoire rapide du GPU.

> Éviter de matérialiser la grande matrice d’attention en mémoire lente.

À RETENIR

On change l’organisation du calcul, sans approximation de parcimonie. Des différences d’arrondi restent possibles.

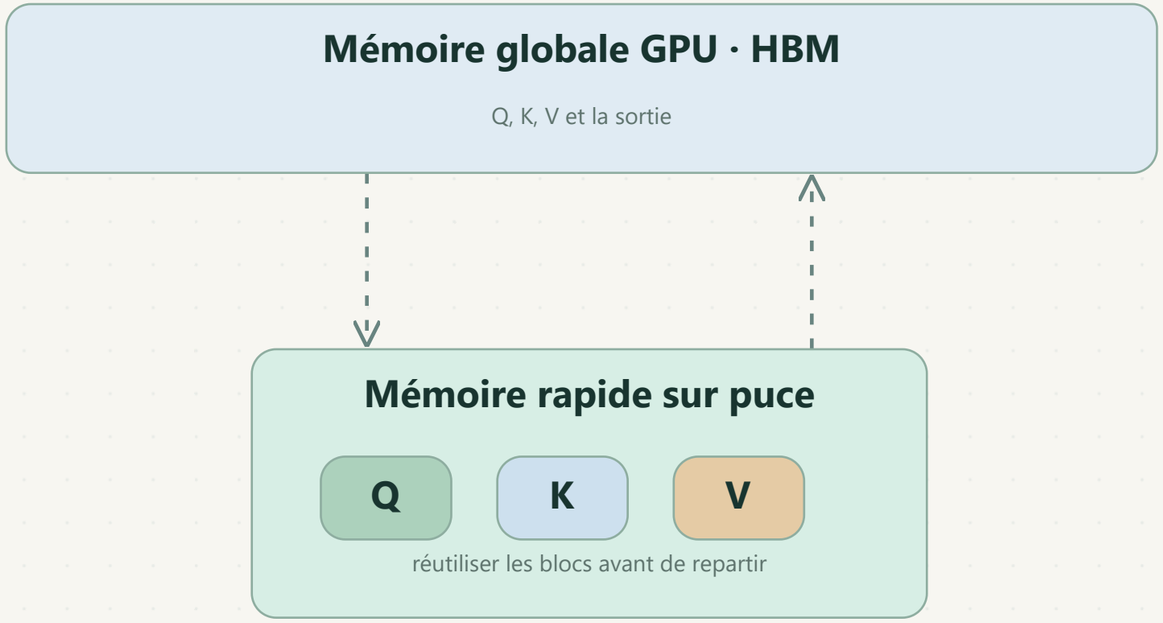


Schéma de principe GPU. FlashAttention ne supprime pas le KV cache et n’accélère pas automatiquement un Raspberry Pi.

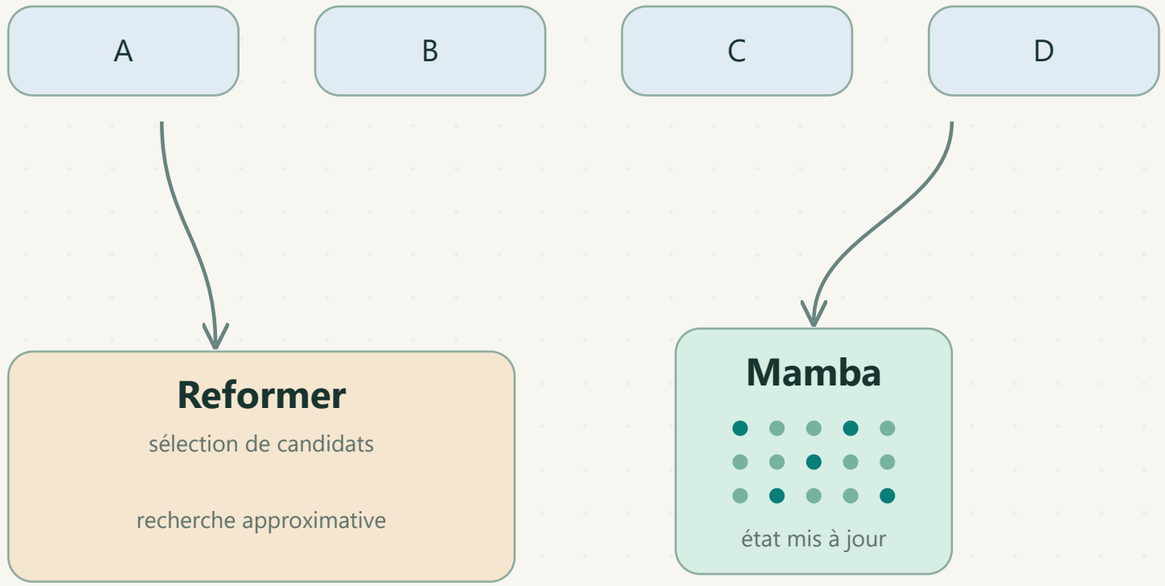
PISTE 3 · CHANGER LA MÉMOIRE DU MODÈLE

Garder des éléments. Ou résumer le passé.

Reformer et Mamba explorent des choix différents de l'attention dense.

> **Reformer · 2020 :** le hachage sélectionne des candidats. Des éléments utiles peuvent être manqués.

> **Mamba · 2023 :** un état récurrent résume l'histoire ; sa taille ne croît pas avec le contexte, pour un modèle fixé.



Choisir une autre représentation change les compromis.

À RETENIR

Un résumé de taille fixe ne garantit pas la restitution exacte de tout le passé. Les compromis se testent tâche par tâche.

Kitaev et al. pour Reformer ; Albert Gu et Tri Dao pour Mamba. Ce ne sont pas des remplacements universellement meilleurs.

PISTE 4 · ENTRAÎNER POUR UNE AUTRE ARITHMÉTIQUE

Et avec des poids −1, 0, +1 ?

BitNet b1.58 apprend avec des poids ternaires. Ce n'est pas simplement arrondir n'importe quel modèle déjà entraîné.

$$(-1)\times x_1 + 0\times x_2 + (+1)\times x_3$$
$$= -x_1 + x_3$$

$\log_2(3) \approx 1{,}58$: contenu théorique d'un symbole ternaire équiprobable. Stockage réel, échelles et autres tenseurs s'ajoutent.

−1	0	+1
soustraire	ignorer	additionner

$y = -x_1 + x_3$

Exemple de poids ternaires ; activations non binaires.

À RETENIR

Des modèles et du code existent. Il faut un entraînement et des noyaux compatibles, puis vérifier la qualité sur sa tâche.

Ma et al., 2024 ; rapport 2B4T, 2025. Ternaire ne veut pas dire que tout calcul devient XOR/POPCNT.

PISTE 5 · PROPOSER, PUIS VÉRIFIER

Un petit modèle écrit. Le grand contrôle.

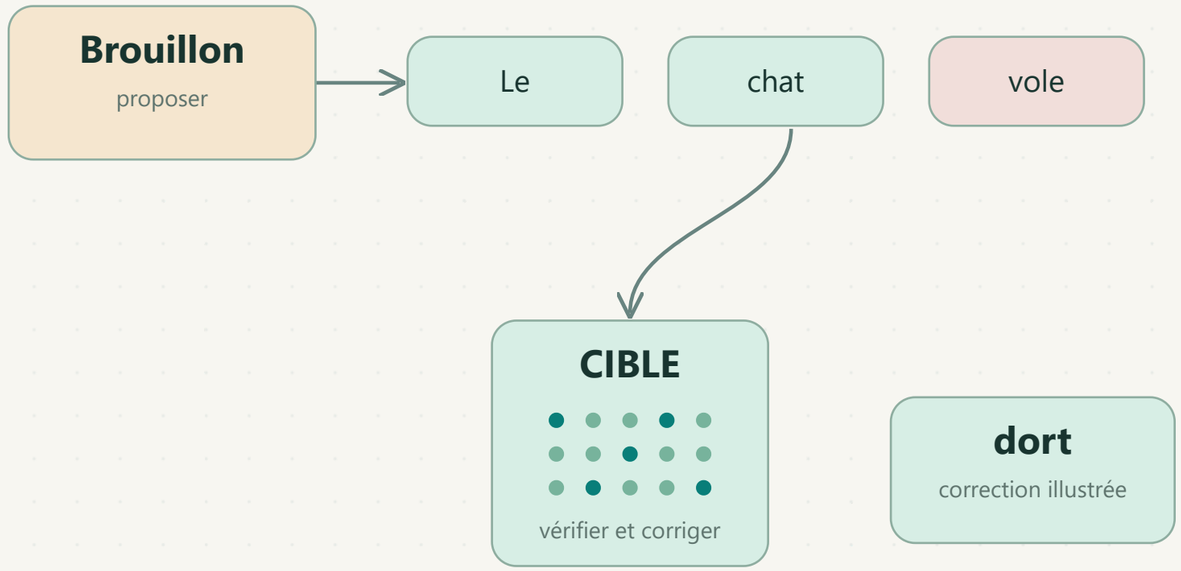
Le décodage spéculatif vérifie ensemble plusieurs tokens proposés par un modèle brouillon.

> Les propositions ne sont pas toutes acceptées.

> Pour l'algorithme exact, une correction d'échantillonnage préserve la distribution du modèle cible.

À RETENIR

Le gain dépend du coût du brouillon, des propositions acceptées et de l'implémentation. Il peut être faible ou nul.



La vraie règle porte sur des probabilités, pas sur « cela semble juste ».

Leviathan, Kalman et Matias, ICML 2023. Ce n'est pas une suppression magique de toute dépendance séquentielle.

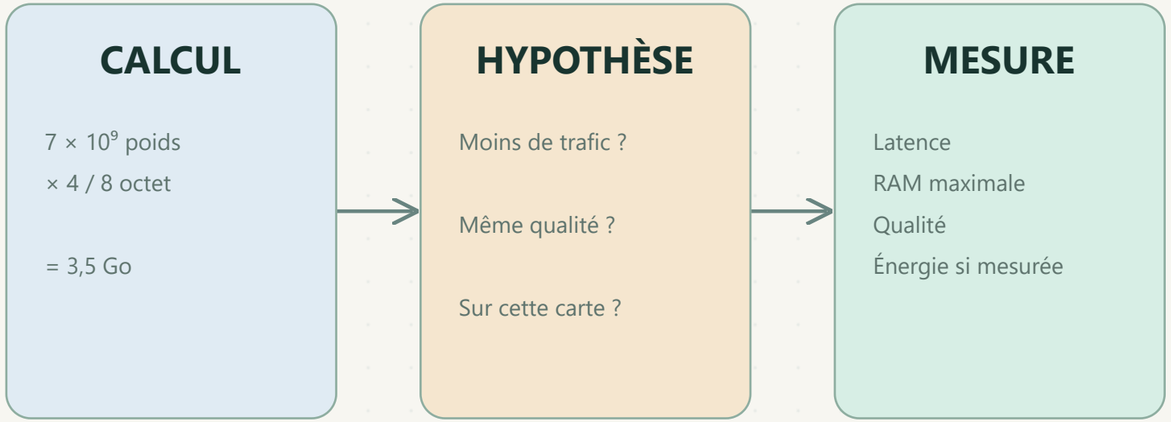
DISTINGUER LES NIVEAUX DE PREUVE

Une équation n’est pas un chronomètre.

Un calcul peut aider à choisir une piste. Il ne démontre pas la qualité d’un modèle ni sa vitesse réelle.

> **Calculé :** le volume des poids, une borne de temps.

> **À mesurer :** latence, RAM maximale, qualité et, si instrumentée, énergie.



Un débit et un score de qualité exigent des expériences.

À RETENIR

Ni « 150 tokens/s » ni « 99,2 % de fidélité » ne découlent des seules équations de la pièce jointe.

Une signature LSH ne contient pas automatiquement les valeurs V. Une architecture proposée doit être définie, implémentée et évaluée.

LA QUESTION QUI ÉVITE UNE FAUSSE CONCLUSION

Deux textes ont le même petit code.

Peut-on garantir qu'on retrouvera toujours l'information exacte avec ce code seul ?

A. Oui : un petit code garde toute l'information.

B. Non : des informations différentes peuvent partager un code.

C. Oui : il suffit d'utiliser XOR.

Un code compact peut avoir des collisions. Il faut définir ce qui est conservé, ce qui est récupéré et comment les erreurs sont détectées.

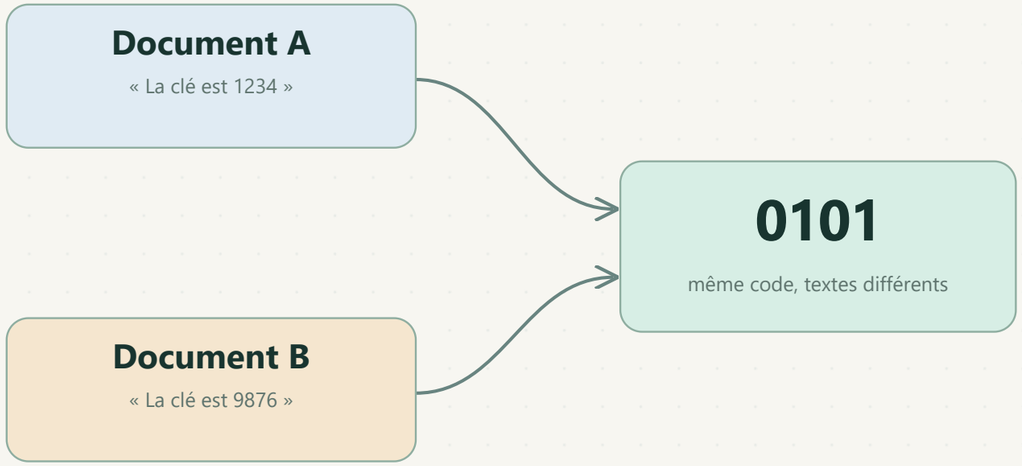


Schéma de collision construit pour expliquer. Il ne mesure pas le taux de collision d'une méthode publiée.

À RETENIR

Un coût réduit doit être accompagné d'une vérification de ce qui a été perdu.

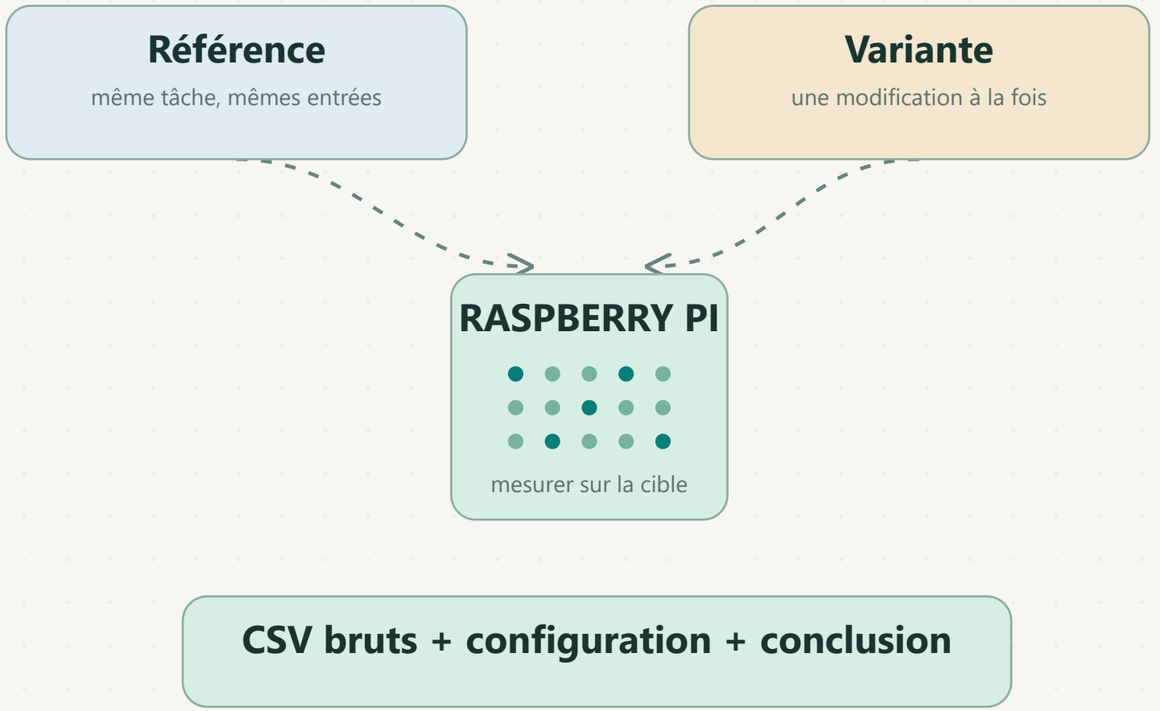
DU PC À LA MACHINE VISÉE

Sur le Pi : même tâche, vraies mesures.

Fixer le matériel, le modèle, le runtime, les données et les critères de qualité avant de comparer.

> Séparer chargement, premier token et génération. Mesurer aussi la RAM maximale.

> Échauffer, répéter, alterner les versions ; garder configuration, CSV et empreintes.



À RETENIR

Une conclusion honnête peut être : « moins de mémoire, même qualité ici, pas de gain de vitesse mesurable ».

Méthode proposée, pas un essai réalisé sur carte dans ce mini-cours. Sans instrument, ne pas annoncer une énergie économisée.

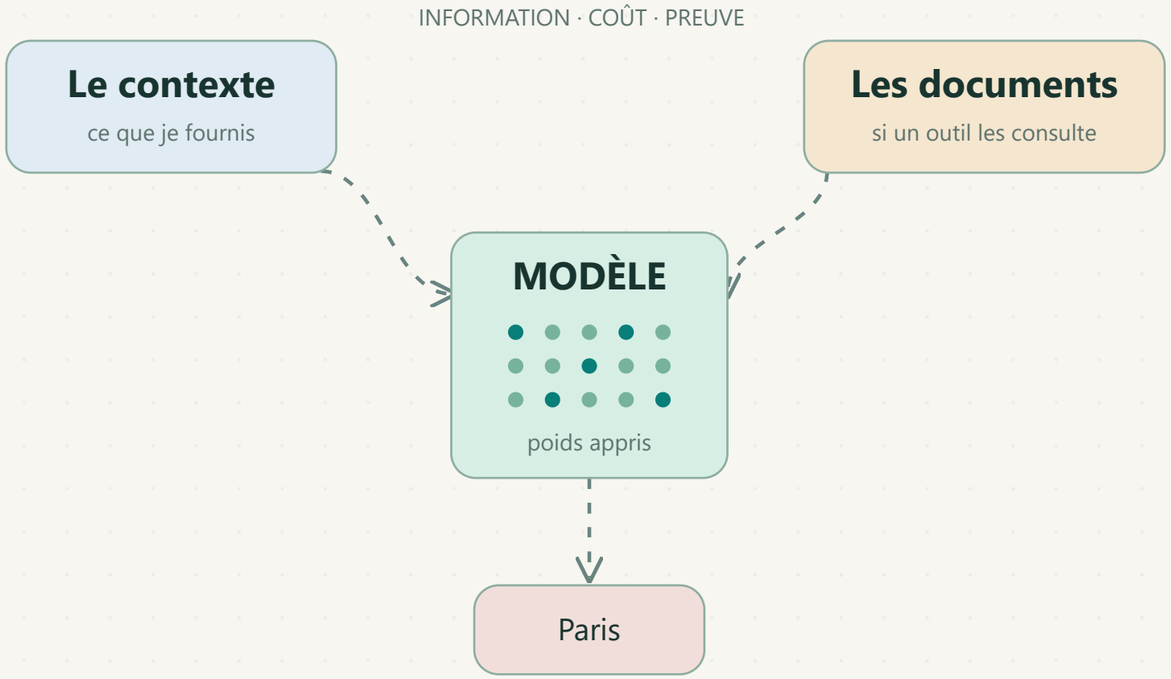
TROIS RÉFLEXES D’INGÉNIEUR

Où est l’information ? Quel est le coût ? Quelle est la preuve ?

Une optimisation utile respecte le besoin de l’utilisateur et les limites de la machine.

> Les poids apprennent ; le contexte oriente ; les documents peuvent compléter.

> Moins de bits, moins de transferts ou une autre architecture : chaque piste demande une vérification.



À RETENIR

Prochain pas : choisir une seule brique, écrire une hypothèse et comparer à une référence.

Les trois pages suivantes donnent les sources et leurs limites. Le cours complet et le TD restent accessibles dans les ressources.

TEXTES ORIGINAUX

Représenter, apprendre, interpréter.

Des sources pour préciser ce que les schémas simplifient.

R1 · Attention Is All You Need ↗

Ashish Vaswani et al. · 2017
NeurIPS 2017

R2 · Locating and Editing Factual Associations in GPT ↗

Kevin Meng, David Bau, Alex Andonian, Yonatan Belinkov · 2022
NeurIPS 2022

R3 · Toy Models of Superposition ↗

Nelson Elhage et al. · 2022
Article de recherche · modèles jouets

R4 · Extracting Training Data from Large Language Models ↗

Nicholas Carlini et al. · 2021
USENIX Security 2021 · prépublication initiale 2020

R6 · GQA: Training Generalized Multi-Query Transformer Models… ↗

Joshua Ainslie et al. · 2023
EMNLP 2023

À RETENIR

Les mécanismes observés dans une expérience ne constituent pas une carte complète de tous les modèles.

TEXTES ORIGINAUX

Compter les octets.
Changer les calculs.

Un gain publié garde le contexte matériel et le protocole du papier.

R5 · Roofline: An Insightful Visual Performance Model… ↗

Samuel Williams, Andrew Waterman, David Patterson · 2008 / 2009
Rapport Berkeley 2008 · article CACM 2009

R7 · AWQ: Activation-aware Weight Quantization… ↗

Ji Lin et al. · 2023 / 2024
Prépublication 2023 · MLSys 2024

R8 · FlashAttention: Fast and Memory-Efficient Exact Attention… ↗

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré · 2022
NeurIPS 2022

R9 · Reformer: The Efficient Transformer ↗

Nikita Kitaev, Łukasz Kaiser, Anselm Levskaya · 2020
ICLR 2020

R10 · Mamba: Linear-Time Sequence Modeling with Selective State Spaces ↗

Albert Gu, Tri Dao · 2023 / 2024
Prépublication 2023 · COLM 2024

À RETENIR

Les simulations de cette présentation sont des exemples pédagogiques ; aucun LLM n’a été exécuté sur Pi ici.

ARTICLES ET IMPLÉMENTATIONS

Des pistes réelles. Des limites à vérifier.

Sélection de publications et de code ; ce n’est pas un classement exhaustif de la recherche.

R11 · The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits ↗

Shuming Ma et al. · 2024
Prépublication · résultats dans les configurations étudiées

R12 · BitNet b1.58 2B4T Technical Report ↗

Shuming Ma et al. · 2025
Rapport technique · prépublication

R13 · Fast Inference from Transformers via Speculative Decoding ↗

Yaniv Leviathan, Matan Kalman, Yossi Matias · 2023
ICML 2023 · prépublication initiale 2022

R14 · GGUF — spécification du format ↗

Projet ggml · Documentation évolutive
Documentation officielle

R15 · BitNet — implémentation officielle ↗

Microsoft Research · Code évolutif
Dépôt officiel

À RETENIR

Revenir aux données et aux conditions d’essai avant de transporter un résultat vers sa propre machine.